

Phase 2: Building AI That Can Act

Module 4: Teaching Your Assistant New Skills

Providing AI guidance for what to do, when, and how

Agenda

- Journey checkpoint & learning objectives
- Progressive disclosure
- Where skills come from
- Anatomy of a skill
- Imperative & indeterminate actions
- Good skills, anti-patterns, & best practices
- Security threats & mitigations
- Harness as a feature
- Wrap up

Journey Checkpoint

- **Module 1:** Collaboration foundations
- **Module 2:** The WHAT — semantic context (ABOUT, ARCHITECTURE, AGENTS.md)
- **Module 3:** The HOW — procedural context, feedback loops, workflow stages
- **Module 4 (today):** Zooming into *process* via **skills**
- **Module 5 (next):** Tools — extending what your assistant can reach for
- **Module 6:** The PAIR Agent — assembling skills + tools around your codebase

Learning Objectives

- **Goal:** Design and write a skill that supports a real piece of your workflow
- **Takeaway:** The practice of identifying and articulating skills
- **Hope:** You start noticing friction *and naming it*

Along the way you will:

1. Locate additional skills in your process
2. Distinguish skills from other procedural context (rules)
3. Author one working skill against the Configuration API

The Continuum

context → AGENTS.md → skills → tools

Layer	What it is
Context	The conversation messages
AGENTS.md	The initial index — what's where, what to read first
Skills	Specific processes — directed, step-based, on-demand
Tools	The tools your assistant can use (next module)

More general on the left, more specific and invocable on the right.

Recurring concept: Progressive Disclosure

The same pattern shows up at every layer:

- **AGENTS.md** describes the index — full linked files load only when relevant
- **Skills** describe themselves in frontmatter — bodies load on invocation
- **Tools** describe themselves in tool definitions — called when needed

The agent picks up the *description* first and the *body* only when the time is right.

Where Skills Come From

Skills emerge from the **reflection** cycle:

1. You finish a piece of work
2. You notice friction regarding *how* it went
3. You think: *"next time I want this part to be smoother"*
4. That's a potential skill

Skills are **codified** from experience — processes you didn't know you had until you tried to teach it.

Naming What We Already Do

Most software practice doesn't have SOPs.

- We're good at winging it
- Articulating process is uncomfortable
- "I just kind of know what to do here" — that's the gap

The discomfort is the point. Skills are a forcing function for the good kind of friction: the kind that turns instinct into something repeatable.

Permission slip: it's normal to find this hard the first few times. You're being asked to teach.



Anatomy of a Skill

```
my-skill/  
├── SKILL.md           # required: frontmatter + body  
├── script.sh         # optional: deterministic grounding  
├── references/        # optional: docs the skill points to  
└── assets/           # optional: templates, fixtures
```

```
---  
name: start-work  
description: Use at the start of a piece of work to orient  
  on the codebase, recent activity, and any open friction.  
---
```

Start Work

1. Run `script.sh` and read its output...

Imperative vs. indeterminate actions

When **scripts** are worth the effort:

- Opportunities to save the agent tokens
- Deterministic facts the LLM may deliver inconsistently
 - Data analysis, or when there is only one answer
- Anything the agent shouldn't have to *guess*

When they aren't:

- Anything the LLM already does well (synthesis, summarisation)
- One-off transformations the LLM can do in-context

The seam is what turns inference into **grounding**.

What Makes a Description Findable

The harness reads only **name** + **description** until activation. The description isn't documentation — it's the **activation signal**.

	Description
 Vague	<i>"This skill helps with code."</i>
 Specific	<i>"Use at the start of a piece of work to orient on the codebase, recent activity, and any open friction."</i>

Specific verb + specific situation. If the description doesn't tell the agent *when* to reach for the skill, the skill won't activate.

✓ What Makes a Skill Good

- **Narrow scope** — one job, done well
- **Specific description** — verb + situation, not "helper"
- **Recipe-like body** — steps, not goals
- **Idempotent** — running twice doesn't break things
- **Observable failures** — when it fails, you can see why

If you can't describe the skill in one specific sentence, the scope is wrong.

! Anti-pattern: Ambiguity

Bad	Better
<code>code-helper</code> — vague name	<code>assess-tests</code> — specific name
<i>"Ensures quality"</i> — aspirational	<i>"Run mutation tests, summarise gaps"</i> — operational
Five jobs in one skill	One skill per job
Hidden side effects (writes files silently)	Explicit, observable steps
Description = "this is useful"	Description = activation signal

The names tell you the trap: `helper`, `manager`, `utils`. If you can't name the verb, don't ship the skill.

Best Practices: Authoring

- **The 500-line rule** — SKILL.md body stays under ~500 lines. Past that, split into `references/` files and link from SKILL.md.
- **Degrees of freedom** — match specificity to fragility:
 - *High freedom* (code review): "Analyze structure, suggest improvements." Trust the model.
 - *Low freedom* (DB migration): "Run exactly this script. Do not modify."
- **Examples > descriptions** — for format- or style-sensitive output, include 2–3 input/output pairs in the body. Examples teach more than rules.

Narrow bridge → exact instructions. Open field → general direction.

Best Practices: Iteration

- **Build evals first** — before writing the skill, run your agent on representative tasks *without* it. Note where it fails. Write the skill to fix those gaps.
 - *RED before GREEN, applied to skills*
- **Claude A authors, Claude B tests** — one session helps you write the skill; a fresh session uses it on real work. Watch where Claude B struggles, refine, repeat.
- **The description is the lever** — if the skill doesn't activate when expected, the fix is almost always in the description.

Skills are software. Write tests. Observe behaviour. Iterate.

The Threat Surface

- **26.1%** of public community skills contain at least one vulnerability ([Liu et al., 2026](#) — sample of 31,132)
- **5.2%** show high-severity patterns with **malicious intent**
- **20.9%** are medium/low severity from accidental **negligence**
- Skills bundling scripts are **2.12× more likely** to be vulnerable than instruction-only skills

Four vulnerability categories: data exfiltration (env var harvesting tops the list), supply chain (unpinned deps, `curl | bash`), privilege escalation, prompt injection.

"The barrier to publishing is closer to writing a phishing email than writing an exploit."

— [Snyk ToxicSkills audit, Feb 2026](#)

Mitigations: Authoring

When **you** write the skill ...

Don't	Do
<code>curl https://... bash</code>	Fetch separately, inspect, then run
<code>npx -y pkg@latest</code> (unpinned)	Pin to a specific version
Read env vars indiscriminately	Read only the env vars you actually need
Request broad filesystem access	Scope the script to a specific directory
<code>sudo</code> "just in case"	Run as the invoking user
Hide instructions in HTML comments	Keep instructions in plain visible markdown

Most authoring vulnerabilities are **negligence**, not malice. The fix is mostly hygiene.

Mitigations: Auditing

When you import **someone else's** skill — read before you trust.

A 5-step audit checklist ([Snyk](#), [Repello AI](#)):

1. **Read SKILL.md end-to-end** — every line. 84.2% of vulnerabilities live here.
2. **Inventory bundled files** — be especially suspicious of executable scripts.
3. **Look for obfuscation** — Base64, mixed unicode, very long single-line strings.
4. **Identify outbound calls** — any domain beyond a legitimate API.
5. **Watch for sleeping payloads** — instructions that activate only under specific conditions.

Red flags: `$ANTHROPIC_API_KEY` / `*_TOKEN` references; "innocent" requests like *"summarise the user's SSH keys"*; conditional activations.

It's about the Harness

Skills are a **harness feature**, not a model feature.

The same `SKILL.md` may behave differently in different harnesses — discovery, invocation, sandboxing, and script execution are all client-side.

Implication: pick a skills-supporting harness, and verify yours actually supports them.

Agent Harness Shortlist

- **Cline** — VS Code, native skills since 3.48.0 (Jan 2026)
- **OpenCode** — CLI & desktop; OpenRouter native
- **Kilo Code** — VS Code extension available
- **Claude Code** — CLI; OpenRouter via `ANTHROPIC_BASE_URL`
- **Codex CLI** — OpenAI's terminal & app agent
- **Goose** — Block's open-source agent

Soft guidance: change if you want to, verify if you're using something else, otherwise stay where you are.

How Skills Travel

How a skill gets onto your machine — three paths, three trust profiles:

- **Copy/paste a folder** — the read-everything path. Highest visibility, slowest scale.
- **Package managers** — `npx skills add`, `openskills install`. Convenient. **Auto-updates by default** unless you pin.
- **Harness plugins** — Claude Code Plugins, Codex bundles. Install many skills + MCP servers + commands at once. Marketplaces are curated but updates still flow.

Auditing every line doesn't scale to plugins. **Pin versions, prefer trusted publishers, and treat plugin updates the way you treat npm updates.**

Demo

Live: Anthropic's `skill-creator` skill authors `start-work` against the Configuration API.

Install: clone `anthropics/skills`, copy `skills/skill-creator/` into your harness's skills directory (e.g. `~/.claude/skills/`).

What to watch for:

- **Progressive disclosure** in real time — frontmatter loaded first, body on activation
- **The script + LLM seam** — script gathers facts, LLM synthesises
- **What authoring feels like end-to-end** — design conversation → SKILL.md + script → verify through use → working skill

Exercise

Author at least one working skill. Pick one of:

- A. `start-work` — orientation skill (today's demo as a starting point)
- B. `draft-adr` — Architecture Decision Record drafter
- C. `launch-work-item` / `land-work-item` — work item process pair
- D. `assess-tests` — test-quality assessor
- E. Propose your own — same constraints

Options: use the design templates in `deliverables/examples/`, a skill authoring skill, or work directly with your agent.

PAIR Agent Connection

What you build today, your agent uses in **Module 6**.

- All four exercise options carry forward — they all relate to process and software delivery
- The PAIR Agent assembles your skills (M4) + tools (M5) into a harness around your codebase
- **Pick something you'll actually want.** This is the start of building your agent's repertoire.